UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA
Escuela Profesional de Ingeniería Estadística e Informática



Universidad
Nacional del
Altiplano de Puno



EJERCICIOS DE PILAS (STACK)

Estructura de Datos

Práctica Académica en C++

Docente:	Prof. Fred Torres Cruz
Estudiante:	Alberto Roy Chambilla Murillo
Semestre Académico:	2026-II
Fecha:	30/10/2026
Repositorio:	Ver código fuente en GitHub

Introducción

Una pila (*Stack*) es una estructura de datos que trabaja bajo el principio **LIFO** (*Last In, First Out*), lo que significa que el último elemento en ingresar es el primero en salir.

En C++, se puede implementar utilizando la biblioteca estándar `<stack>` y sus operaciones principales: `push()`, `pop()`, `top()` y `empty()`.

Puedes acceder al repositorio de código fuente completo en el siguiente enlace:

https://github.com/albertoroy3210-creator/Ejercicios-de-estructura/tree/main/Trabajo%202/Codigo2_Pdf_Latex

---

1. Ejercicio 1: Apilar números enteros

Objetivo: Ingresar cinco números enteros, almacenarlos en una pila y retirarlos uno por uno hasta dejarla vacía.

Código en C++

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> pila;
    int numero;

    for (int i = 0; i < 5; i++) {
        cout << "Ingrese un numero: ";
        cin >> numero;
        pila.push(numero);
    }

    cout << "\nElementos desapilados: ";

    while (!pila.empty()) {
        cout << pila.top() << " ";
        pila.pop();
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & C:\Users\USER\Downloads\5.exe
Ingrese un numero: 5
Ingrese un numero: 1
Ingrese un numero: 2
Ingrese un numero: 3
Ingrese un numero: 6

Elementos desapilados: 6 3 2 1 5
```

Figura 1: Ejecución del Ejercicio 1

Ejemplo de ejecución

```
Entrada: 4, 8, 12, 16, 20
Salida: 20, 16, 12, 8, 4
```

Explicación: Los números se insertan mediante `push()`. Posteriormente, se utiliza `top()` para consultar el elemento superior y `pop()` para eliminarlo. De esta manera, se obtiene el orden inverso de inserción.

2. Ejercicio 2: Invertir una palabra

Objetivo: Leer una palabra, almacenar cada uno de sus caracteres en una pila y mostrarla en orden inverso.

Código en C++

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    stack<char> pila;
    string palabra;

    cout << "Ingrese una palabra: ";
    cin >> palabra;

    for (char letra : palabra) {
        pila.push(letra);
    }

    cout << "Palabra invertida: ";

    while (!pila.empty()) {
        cout << pila.top();
        pila.pop();
    }

    return 0;
}
```

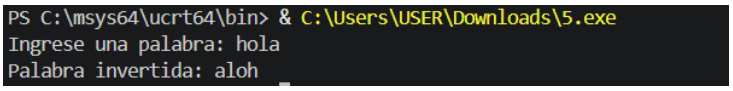


Figura 2: Ejecución del Ejercicio 2

Ejemplo de ejecución

```
Entrada: DATOS
Salida: SOTAD
```

Explicación: La pila permite invertir una palabra porque el último carácter insertado es el primero en salir, cumpliendo el principio LIFO.

3. Ejercicio 3: Verificar paréntesis balanceados

Objetivo: Determinar si los paréntesis de una expresión están correctamente balanceados.

Código en C++

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    stack<char> pila;
    string expresion;
    bool correcto = true;

    cout << "Ingrese una expresion: ";
    getline(cin, expresion);

    for (char caracter : expresion) {
        if (caracter == '(') {
            pila.push(caracter);
        }
        else if (caracter == ')') {
            if (pila.empty()) {
                correcto = false;
                break;
            }
            pila.pop();
        }
    }

    if (!pila.empty()) {
        correcto = false;
    }

    if (correcto) {
        cout << "Los parentesis estan balanceados.";
    }
    else {
        cout << "Los parentesis NO estan balanceados.";
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & C:\Users\USER\Downloads\5.exe
Ingrese una expresion: 23+9=x
Los parentesis estan balanceados.
```

Figura 3: Ejecución del Ejercicio 3

Ejemplos de ejecución

```
Entrada: (a+b)*(c-d)
Salida: Los parentesis estan balanceados.

Entrada: ((a+b)
Salida: Los parentesis NO estan balanceados.
```

Explicación: Cada paréntesis de apertura se almacena en la pila. Cuando aparece uno de cierre, se elimina una apertura pendiente. Si no existe una apertura disponible o quedan elementos al finalizar, la expresión no está balanceada.

4. Ejercicio 4: Historial de acciones (Undo)

Objetivo: Simular la función deshacer de un editor, almacenando las acciones realizadas en una pila.

Código en C++

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    stack<string> pila;
    int opcion;
    string accion;

    do {
        cout << "\n--- HISTORIAL DE ACCIONES ---\n";
        cout << "1. Registrar accion\n";
        cout << "2. Deshacer accion\n";
        cout << "3. Mostrar accion en el tope\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;
        cin.ignore();

        switch (opcion) {
            case 1:
                cout << "Ingrese la accion: ";
                getline(cin, accion);
                pila.push(accion);
                cout << "Accion registrada.\n";
                break;

            case 2:
                if (!pila.empty()) {
                    cout << "Deshaciendo: " << pila.top() << endl;
                    pila.pop();
                }
                else {
                    cout << "No hay acciones para deshacer.\n";
                }
                break;

            case 3:
                if (!pila.empty()) {
                    cout << "Accion en el tope: " << pila.top() << endl;
                }
                else {
                    cout << "El historial esta vacio.\n";
                }
                break;

            case 4:
```

```
                cout << "Saliendo del programa.\n";
                break;

            default:
                cout << "Opcion no valida.\n";
        }

    } while (opcion != 4);

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & C:\Users\USER\Downloads\5.exe

--- HISTORIAL DE ACCIONES ---
1. Registrar accion
2. Deshacer accion
3. Mostrar accion en el tope
4. Salir
Seleccione una opcion: 3
El historial esta vacio.
```

Figura 4: Ejecución del Ejercicio 4

Ejemplo de ejecución

```
Registrar: Escribir texto
Registrar: Copiar
Registrar: Eliminar

Deshacer: Eliminar
```

Explicación: Cada acción se almacena en la pila. Al seleccionar la opción deshacer, se muestra la última acción mediante `top()` y se elimina con `pop()`.

5. Ejercicio 5: Conversión de decimal a binario

Objetivo: Convertir un número decimal positivo a binario utilizando una pila para almacenar los residuos de las divisiones sucesivas entre dos.

Código en C++

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> pila;
    int numero;

    cout << "Ingrese un numero decimal positivo: ";
    cin >> numero;

    if (numero < 0) {
        cout << "Ingrese un numero positivo.";
        return 0;
```

```
    }

    if (numero == 0) {
        cout << "Binario: 0";
        return 0;
    }

    while (numero > 0) {
        pila.push(numero % 2);
        numero = numero / 2;
    }

    cout << "Numero en binario: ";

    while (!pila.empty()) {
        cout << pila.top();
        pila.pop();
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & C:\Users\USER\Downloads\5.exe
Ingrese un numero decimal positivo: 10
Numero en binario: 1010
PS C:\msys64\ucrt64\bin>
```

Figura 5: Ejecución del Ejercicio 5

Ejemplo de conversión

Para convertir el número decimal 13 a binario:

División	Cociente	Residuo
13 / 2	6	1
6 / 2	3	0
3 / 2	1	1
1 / 2	0	1

```
Entrada: 13
Residuos apilados: 1, 0, 1, 1
Salida: 1101
```

Explicación: Los residuos de las divisiones se almacenan en la pila. Al retirarlos mediante `pop()`, se obtiene el orden correcto del número binario.

Pregunta de cierre

¿Qué característica común tienen los cinco problemas que hace apropiado utilizar una pila?

Los cinco problemas utilizan el principio **LIFO** (*Last In, First Out*), que establece que el último elemento en ingresar es el primero en salir.

Esta característica permite invertir palabras, recuperar la última acción realizada, verificar paréntesis balanceados, retirar números en orden inverso y convertir números decimales a binarios mediante el almacenamiento ordenado de residuos.

Conclusión

La realización de estos ejercicios permite comprender el funcionamiento de las pilas y aplicar sus operaciones básicas en diferentes problemas de programación. Se demuestra que el principio LIFO resulta sumamente útil cuando se necesita procesar elementos en orden inverso al que fueron almacenados.